

Learning to Play Tak: A comparison of AlphaZero and Proximal Policy Optimization on a Novel Board Game Domain

Sofia Lanzi, Rituraj Navindgikar

Northeastern University

Abstract

We apply two deep reinforcement learning algorithms AlphaZero and Proximal Policy Optimization (PPO) to the board game Tak, a two-player perfect-information strategy game with a large action space and sparse rewards. Both agents are trained from scratch through self-play on a custom 5x5 Tak environment with no domain knowledge beyond the rules. We evaluate each agent against a random baseline, a hand-crafted Alpha-Beta minimax agent, and Tiltak, a strong open-source Tak engine based on Monte Carlo Tree Search. AlphaZero achieves 78.8% win rate against random play and 30.4% against Tiltak, while PPO achieves 70.8% against random and 5.9% against Tiltak. Neither agent defeats the minimax baseline. Our results suggest that PPO's lack of lookahead search is a fundamental limitation in this domain, while AlphaZero's MCTS gives it meaningful strategic depth despite limited training compute. We analyze training dynamics, identify key failure modes, and discuss architectural improvements that could close the performance gap with domain-specific engines.

RL Code - <https://github.com/rituraj-navindgikar/tak-game>

Sim - <https://github.com/akshaykgupta/Tak-sim>

Introduction

Tak is a two-player abstract strategy board game designed by James Ernest and Patrick Rothfuss. Unlike chess or Go, Tak has received almost no attention from the reinforcement learning community, making it an interesting and relatively unexplored domain. The game is played on an NxN grid where players place and move stacks of flat stones, walls, and capstones, with the goal of building a connected road from one side of the board to the other. On a 5x5 board, the action space contains over 400 legal moves per position, games can last up to 150 moves, and rewards are entirely sparse – a player receives no signal until the game ends.

These properties make Tak a challenging testbed for RL. The large branching factor stresses search-based methods, the long horizon with sparse rewards makes credit assignment difficult for policy gradient methods, and the

absence of prior RL work means there is no established training protocol to build on.

In this project we implement and compare two fundamentally different approaches. AlphaZero [1] combines a deep neural network with Monte Carlo Tree Search (MCTS) and trains entirely through self-play, iteratively improving both the policy and value estimates. PPO [2] is a model-free policy gradient method that optimizes a clipped surrogate objective and trains on batches of self-play experience without any lookahead search. The contrast between these two approaches – search-augmented versus purely reactive is particularly interesting in a domain where lookahead depth is critical.

Our contributions are: (1) a complete custom Tak environment and action space implementation, (2) from-scratch implementations of AlphaZero with MCTS self-play and PPO with GAE, (3) a suite of evaluation agents including a minimax Alpha-Beta player and integration with the Tiltak engine, and (4) an empirical comparison of both algorithms across 1000-game evaluation runs with analysis of training dynamics and failure modes.

Background

2.1 The Game of Tak

Tak is played on a 5x5 board. Each player begins with 21 flat stones and 1 capstone. On the first two moves, each player places an opponent's flat stone (the opening swap rule). Subsequently, a player may either place a stone from their reserve or move a stack of stones already on the board. Stacks can be moved in the four cardinal directions, dropping stones along the path. A wall (standing stone) blocks stack movement but can be flattened by a capstone. The game ends when a player completes a road a path of their flat stones or capstones from one side of the board to the opposite side or when all squares are filled or a player exhausts their stones, in which case the player with more flat stones on top of stacks wins.

For this project we implement the no-capstone variant on a 5x5 board, giving each player 21 flat stones and no

capstones. This simplifies the action space while preserving the core road-building strategic challenge.

2.2 AlphaZero

AlphaZero [1] is a general self-play reinforcement learning algorithm that masters games from random initialization with no domain knowledge beyond the rules. A deep neural network θ takes a board state s as input and outputs a policy vector p over all legal moves and a scalar value $v \in [-1, 1]$ estimating the probability of winning from state s . During self-play, MCTS uses the network’s policy to guide tree search and the value to evaluate leaf nodes, producing an improved policy π based on visit counts. The network is then trained to minimize:

$$L = (z - v)^2 - \pi^T \log p + c \|\theta\|^2$$

Where $z \in \{-1, 0, 1\}$ is the game outcome. This iterative process of self-play data generation followed by network training gradually improves the quality of both search and evaluation.

2.3 Proximal Policy Optimization

PPO [2] is a model-free policy gradient algorithm that stabilizes training through a clipped surrogate objective. Given a policy π_θ and an old policy $\pi_{\theta_{old}}$, the probability ratio $r_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{\theta_{old}}(a_t | s_t)$ is clipped to $[1 - \epsilon, 1 + \epsilon]$ to prevent destructive updates:

$$L^{clip} = E_t [\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t)]$$

Advantage estimates $\hat{A}_t$ are computed using Generalized Advantage Estimation (GAE). Unlike AlphaZero, PPO selects moves greedily from the policy network without any tree search, making it faster per move but blind to future consequences.

2.4 Baseline Agents

- **Random Agent:** Selects uniformly at random from all legal moves. Serves as a lower bound on performance.
- **Minimax (Alpha-Beta):** A hand-crafted depth-limited minimax search with alpha-beta pruning. The evaluation function counts flat stone differential and connectivity. This represents classical game-playing AI without learning.
- **Tiltak:** An open-source Tak engine that uses MCTS with domain-specific heuristics. Tiltak represents a strong, specialized opponent. We run Tiltak in a no-capstone configuration to match our environment.

Related Work

AlphaZero has been successfully applied to chess, shogi, and Go [1], and has inspired numerous implementations on other games. General board game learning frameworks such as the GBG framework [3] have applied AlphaZero-inspired methods to Othello, Connect Four, and Hex. However, to our knowledge, no prior work has applied AlphaZero or PPO to Tak specifically, making this an unexplored domain.

The closest related work is on games with similar properties to Tak: large branching factors, stacking mechanics, and road-building objectives. Hex has been studied with neural MCTS approaches and shares the road connectivity win condition with Tak. Prior work on resource-constrained AlphaZero [4] notes that the significant computational demands of AlphaZero are a barrier to reproducibility on non-standard hardware, which is directly relevant to the limitations of our training setup. PPO has been widely applied to board games as a simpler baseline against search-based methods. The general finding in the literature is that model-free methods without lookahead struggle in games requiring deep strategic planning, while MCTS-augmented methods are more sample-efficient in such domains.

Project Description

4.1 Environment

We implement a complete 5x5 Tak environment in Python. The state is encoded as a 129-dimensional vector: 5 features per cell (top stone owner, top stone type, stack height, fraction of P0 stones, fraction of P1 stones) across all 25 cells, plus 4 global features (current player, P0 flat count, P1 flat count, move counter). The action space contains all legal place and move actions, enumerated at initialization, with illegal moves masked during inference.

The environment implements the full Tak rule set including stack movement with arbitrary drop sequences, wall placement, road detection via DFS, and flat stone count tiebreaking. The no-capstone variant removes wall-flattening mechanics, simplifying stack interactions.

4.2 AlphaZero Implementation

The AlphaZero network is a 3-layer MLP with LayerNorm and ReLU activations, hidden dimension 256, with separate policy and value heads. The policy head outputs logits over all 400+ actions; illegal moves are masked to $-\infty$ before softmax. The value head uses Tanh activation to output $v \in [-1, 1]$

MCTS uses the PUCT selection rule with $C_{puct} = 1.5$ and Dirichlet noise ($\alpha=0.3$, weight 0.25) added at the root during self-play for exploration. Each self-play game uses temperature $\tau=1.0$ for the first 10 moves and $\tau=0$ thereafter.

The reward signal combines game outcome with road progress shaping:

$$z = 0.9 * z_{outcome} + 0.1 * \Delta_{road}$$

Where Δ_{road} is the difference in largest connected flat stone components, normalized by board size. This provides a small dense signal to guide early learning.

Training runs for 100 iterations. Each iteration generates 100 self-play games using 16 parallel CPU workers with 50 MCTS simulations per move, trains for 20 epochs on a replay buffer capped at 100,000 positions (batch size 512), and evaluates the new checkpoint against the previous best.

The optimizer is Adam with learning rate 10^{-4} and weight decay 10^{-4} .

Rituraj Navindgikar implemented the AlphaZero agent, MCTS, training pipeline, and evaluation framework.

4.3 PPO Implementation

The PPO network shares the same 3-layer MLP architecture (hidden dimension 256, LayerNorm, ReLU) with separate policy and value heads, but the value head uses no activation function (unbounded output).

PPO trains for 300 iterations. Each iteration collects 128 self-play games using 4 parallel workers, then performs 16 epochs of PPO updates with batch size 512.

Key hyperparameters: learning rate 10^{-4} , discount factor $\gamma=0.995$, GAE lambda $\lambda=0.97$, clip epsilon $\epsilon=0.15$, entropy coefficient 0.005, value loss coefficient 0.5, and gradient norm clipping at 0.5. The longer training horizon (300 vs 100 iterations) compensates for PPO's lower per-iteration sample quality.

During inference, PPO selects moves greedily by argmax over masked logits with no tree search.

Sofia Lanzi implemented the PPO agent and training pipeline. Both authors contributed to the game environment and evaluation framework.

Experiments

5.1 Training Dynamics

AlphaZero (Figure 1): Policy loss drops from 4.78 at initialization to approximately 2.77 by iteration 5,

reflecting rapid early learning, then gradually rises to a plateau around 3.0 for the remainder of training. Value loss remains stable near 0.09 throughout. The policy loss plateau at 3.0 corresponds to the network distributing probability across roughly 20 actions out of 400+, indicating the model has learned to prefer a subset of plausible moves but has not developed strong positional preferences. Average reward per game remains near 0.006 throughout training, reflecting the sparsity of the reward signal. Self-play game outcomes show no systematic first-mover advantage, with P0 and P1 wins alternating roughly evenly.

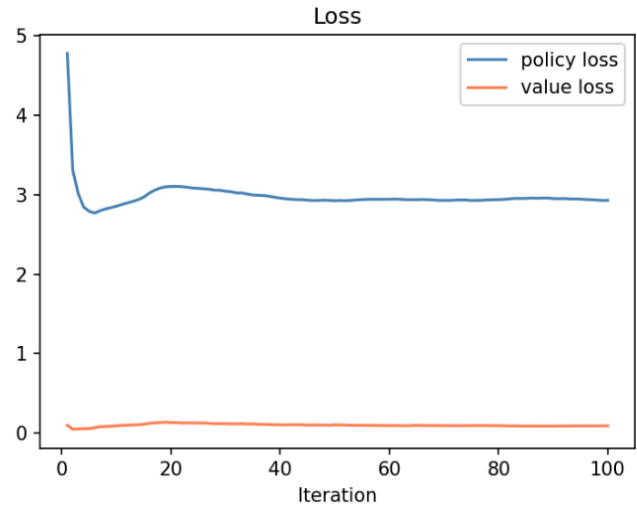


Figure 1.1 Policy loss and value loss of the AlphaZero agent over 100 self-play iterations

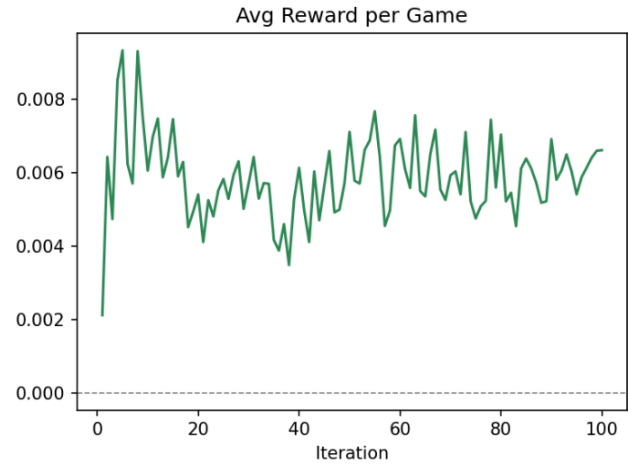


Figure 1.2: AlphaZero average reward per game

PPO (Figure 2): Policy loss decreases steadily and approaches near-zero by iteration 300, indicating the policy has converged to near-deterministic move selection. Value loss decreases from approximately 0.20 to near-zero. Entropy begins high and gradually decreases as the policy sharpens, with notable spikes around iteration 130 suggesting the agent periodically rediscovers exploratory

strategies. Most significantly, average reward per game shows a clear upward trend from near zero to approximately 0.10 by iteration 300, indicating the agent is genuinely improving its ability to win games over the course of training. Game outcomes show a sharp regime change around iteration 70 where P0 wins become dominant, then another shift around iteration 130 – reflecting the PPO agent discovering and converging on effective opening strategies.

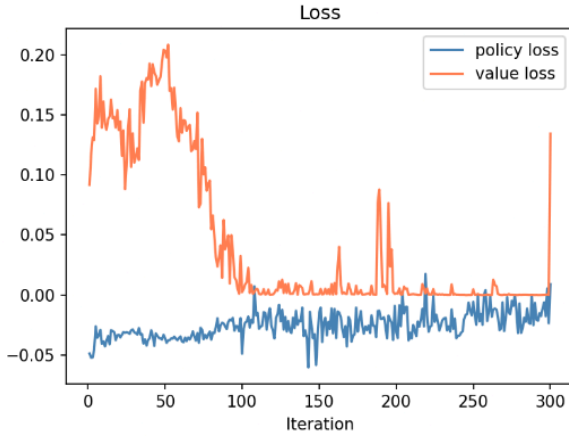


Figure 2.1: Policy loss and value loss of PPO agent over 300 self-play iterations

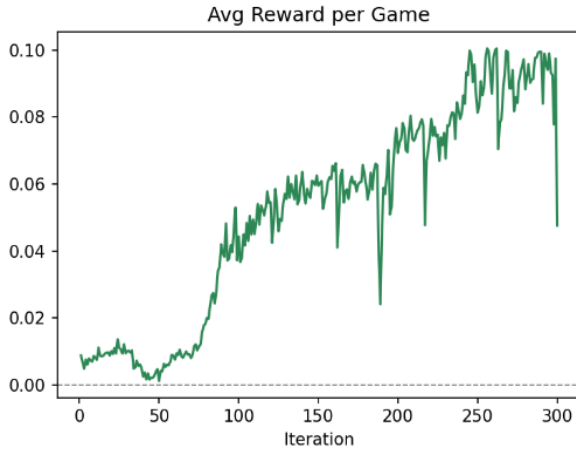


Figure 2.2: PPO average reward per game

5.2 Evaluation Results

All evaluations alternate first-mover positions across games to control for first-move advantage.

The near-identical win rates across conditions confirm that board position does not systematically favor either player in our 5x5 no-capstone environment, making alternating evaluation a fair and sufficient control. Results are summarized in Table 1.

Matchup	Games	Agent 1 Win %	Agent 2 Win %	Draw %
AlphaZero v/s Random	1000	78.8%	20.3%	0.9%
PPO v/s Random	1000	70.8%	28.9%	0.3%
AlphaZero v/s Minimax	1000	0.0%	100%	0.0%
PPO v/s Minimax	1000	0.0%	100%	0.0%
AlphaZero v/s Tiltak	1000	30.4%	69.6%	0.0%
PPO v/s Tiltak	1000	5.9%	94.1%	0.0%
Minimax v/s Tiltak	1000	78.3%	21.7%	0.0%

Against Random: Both agents convincingly beat random play. AlphaZero (78.8%) outperforms PPO (70.8%) by approximately 8 percentage points, suggesting MCTS provides meaningful strategic depth even at 50 simulations per move. The small draw rate (~0.9% for AZ) reflects rare flat-count tiebreak situations.

Against Minimax: Both agents lose every game to the minimax baseline, regardless of first-mover position. This result is striking when the Alpha-Beta search with even a simple evaluation function completely dominates both learned agents. This reflects a fundamental limitation: without explicit lookahead, PPO cannot anticipate threats more than one move ahead, and AlphaZero’s 50 MCTS simulations are insufficient to search deeply enough to counter a dedicated tree search.

Against Tiltak: AlphaZero wins 30.4% of games against Tiltak, a non-trivial result given that Tiltak is a specialized engine with domain-specific heuristics. PPO wins only 5.9%. The gap between the two RL agents is substantial here AlphaZero’s MCTS allows it to occasionally find tactical sequences that exploit Tiltak’s weaknesses, while PPO’s greedy policy is too myopic to construct multi-move threats. Note that Tiltak is run without capstones to match our environment, which partially handicaps it relative to its full-strength configuration.

Minimax vs. Tiltak calibration: Our minimax agent wins approximately 78% of games against Tiltak regardless of move order, confirming that our minimax implementation

is strong and that Tiltak’s performance in our no-capstone environment is significantly below its full-strength rating. This calibration run establishes that the RL agents are being evaluated against a genuinely weaker version of Tiltak, and their performance should be interpreted accordingly.

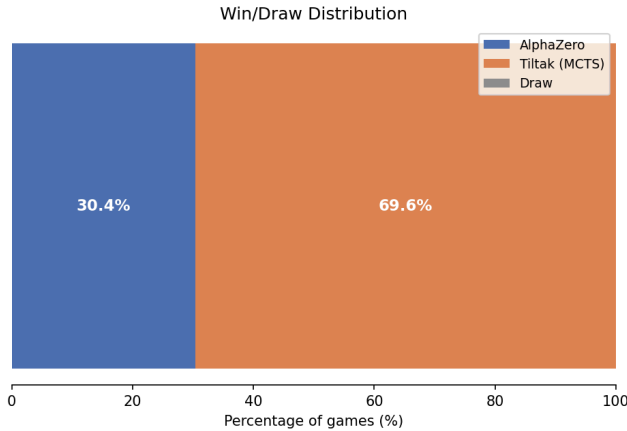


Figure 3: AlphaZero v/s Tiltak over 1000 games

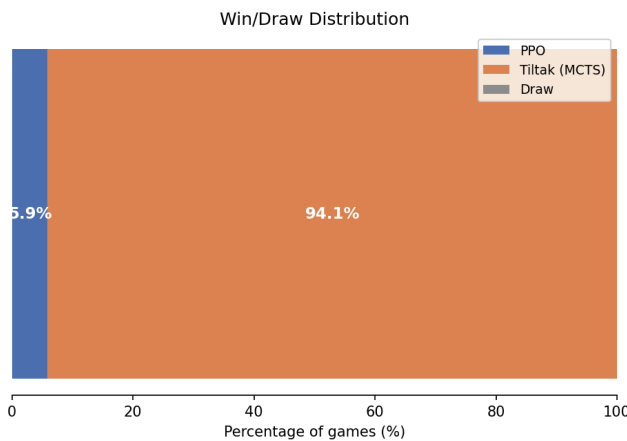


Figure 4: PPO v/s Tiltak over 1000 games

5.3 Analysis

The results reveal a clear performance hierarchy:

Minimax > AlphaZero > PPO > Random

with Tiltak sitting between Random and PPO in our constrained environment.

The key finding is that search depth matters more than learning quality in Tak at this training scale. Both RL agents have learned to beat random play, confirming that self-play training produces non-trivial strategies. However, neither has learned strategies robust enough to counter explicit tree search. AlphaZero’s advantage over PPO is entirely attributable to MCTS – at inference time,

AlphaZero runs 50 MCTS simulations per move, giving it a short lookahead that PPO entirely lacks.

The policy loss plateau at ~ 3.0 for AlphaZero suggests the network has not converged to a sharp, confident policy. With 400+ possible actions and only 50 MCTS simulations, the training signal is too noisy for the network to develop strong positional intuitions. The original AlphaZero used 800 simulations and millions of self-play games; our resource-constrained implementation uses $\sim 10,000$ games with 50 simulations.

PPO’s reward trend is actually more encouraging than AlphaZero’s training curves suggest – the steady increase in average reward indicates genuine learning, while AlphaZero’s reward signal remains flat throughout training. However, PPO’s greedy inference creates a ceiling: without search, even a well-trained policy cannot counter multi-move tactical threats.

Conclusion

We implemented and evaluated AlphaZero and PPO on the board game Tak, a novel domain with no prior RL work. Both agents successfully learn to defeat random play through self-play training, with AlphaZero (78.8%) outperforming PPO (70.8%). Neither agent defeats the minimax baseline, and AlphaZero substantially outperforms PPO against the Tiltak engine (30.4% vs 5.9%), attributable to MCTS lookahead at inference time.

The core lesson from this project is that in a game with a large branching factor and sparse rewards, search depth at inference time is more important than the quality of the learned policy at our training scale. PPO trains faster and shows cleaner convergence, but its lack of lookahead is a fundamental limitation. AlphaZero’s MCTS provides meaningful strategic depth despite a policy that has not fully converged.

Several directions could improve these results. Increasing MCTS simulations from 50 to 800 would substantially improve AlphaZero’s strategic depth, at the cost of training time. A convolutional architecture over the 5x5 board would better capture spatial structure than the flat MLP. Incorporating capstones and the full rule set would make the environment more strategically rich and potentially provide clearer learning signals. Finally, curriculum learning starting against random and gradually increasing opponent strength could address the sparse reward problem in early training.

This project confirms that Tak is a challenging and interesting domain for RL research, and that the performance gap between learned and hand-crafted agents remains substantial at practical compute budgets.

References

- [1] D. Silver et al., "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play," *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018.
- [2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [3] J. Scheiermann and W. Konen, "AlphaZero-like learning in the general board game framework GBG," *arXiv preprint arXiv:2205.01048*, 2022.
- [4] K. Ota et al., "Resource-constrained AlphaZero for combinatorial optimization," 2025.